Is the Hierarchy Doing Anything? Main-Network Collapse in Hierarchical Byte-Level Language Models

Anonymous Authors

Abstract

Hierarchical byte-level language models such as H-Net compress a byte sequence into chunks, process the chunks with a large *main network*, and decode back to bytes, with a residual path from encoder to decoder. We show that this design has a silent failure mode: the main network can *collapse* to an almost input-independent output while the encoder–residual–decoder path alone does the language modelling. We measure this with a simple diagnostic, CTX_GAIN: the increase in validation bits-per-byte (bpb) when the main network's output is replaced by its per-sequence mean. In 80 GPU training runs of 100–150M-parameter models on FineWeb-Edu bytes, both in our re-implementation and in the official H-Net code, every model with static or non-learned chunk boundaries collapsed (CTX_GAIN≤ 0.02), including models with *random* boundaries, while H-Net's learned, content-dependent chunking usually kept the main network in use. In this regime H-Net's advantage over fixed-stride chunking (0.11 bpb in the official code) is explained by *whether the hierarchy is used at all*, and bpb correlates strongly with CTX_GAIN across runs ($r = -0.91$). Collapse is decided within the first 1,000 training steps and never reverses, and it also happens to H-Net itself (3/20 seeds of our re-implementation and 1/9 seeds of the official code, plus one partial collapse costing 0.1 bpb). Three straightforward anti-collapse interventions did not revive static chunking, and none of six new chunking mechanisms we designed beat H-Net. A routing feature that anticipates next-byte difficulty avoided collapse in all 26 of its runs (H-Net: 4/29 collapsed), but this difference is not significant ($p = 0.11$) and the feature does not improve H-Net when H-Net does not collapse. We recommend reporting CTX_GAIN (or an equivalent bypass check) and multiple seeds for any hierarchical byte-level model.

1 Introduction

Tokenizer-free language models operate directly on bytes and recover efficiency by hierarchy: an inexpensive *encoder* works at byte resolution, a large *main network* works on a shorter sequence of chunks, and a *decoder* maps chunk-level states back to byte predictions [7, 9, 11, 13, 14]. H-Net [7] learns where to place chunk boundaries end-to-end ("dynamic chunking") and reports that this outperforms fixed or heuristic chunking.

We set out to design a better dynamic chunking mechanism and instead found that, at the scale we could afford, the decisive variable was not *where* chunks are placed but *whether the main network is used at all*. Hierarchical byte LMs contain a residual path from encoder to decoder; when the encoder and decoder can model local structure themselves, the main network can degenerate into an approximately constant vector without the training loss revealing it. The model then looks like a normal, reasonably good byte-level LM.

Contributions.

- A cheap diagnostic, CTX_GAIN (Section 3), and a complementary inference-time test (changing the compression of a trained model), which expose a bypassed hierarchy.
- Evidence from a re-implementation and from the official H-Net code that static chunking collapses in every run, while H-Net's learned routing usually prevents collapse; H-Net's advantage over fixed-stride chunking tracks main-network use (Section 5).
- Two observations about the mechanism: content-independent *random* boundaries collapse exactly like fixed ones, and the outcome is fixed within the first 1,000 steps (Section 5.3).

- Seed-level statistics (20 seeds per arm in our re-implementation, 9 official H-Net seeds) showing that collapse is stochastic even for H-Net.
- Negative results for six new chunking mechanisms and three anti-collapse interventions, and a null result (26 runs, two implementations) for an anticipatory routing feature (Sections 5.4–5.5).

2 Background: H-Net and dynamic chunking

A one-stage H-Net [7] maps bytes $x_{1:L}$ to encoder states $h_{1:L}$. A routing module predicts boundary probabilities $p_t = \frac{1}{2}(1 - \cos(W_q h_{t-1}, W_k h_t))$; positions with $p_t \geq 0.5$ start a new chunk, and their states form the main network's input sequence $z_{1:M}$ ($M \approx L/N$ for a target compression N, encouraged by a load-balancing loss). The main network's outputs $y_{1:M}$ are *de-chunked* with an exponential moving average whose rate is p, broadcast back to bytes, multiplied by a straight-through confidence term [1], added to a residual projection of h, and decoded. The encoder and decoder use Mamba-2 [4] layers in the official release; the main network is a Transformer. Gradients reach the router only through the EMA rate, the straight-through term and the load-balancing loss.

3 Diagnosing a bypassed hierarchy

CTX_GAIN. For a trained model, we evaluate validation bpb twice on the same batches: normally, and with the main network's output $y_{1:M}$ replaced by its mean over the M chunks of each sequence (keeping its average contribution, removing all chunk-specific information). We define

$$\text{CTX_GAIN} = bpb_{\text{mean-replaced}} - bpb_{\text{normal}}.$$

A model whose decoder ignores the main network's *content* has CTX_GAIN ≈ 0, even if the main network contributes a useful constant bias (zeroing y instead would conflate the two). The test costs two evaluation passes and needs no retraining. We call a run *collapsed* when CTX_GAIN ≤ 0.05; the threshold is arbitrary, but the distribution is clearly separated in practice (Figure 1).

Changing the compression after training. A second, independent signature: if the main network is bypassed, the model's bpb should be insensitive to how bytes are chunked at inference. We vary the stride (fixed chunking) or the boundary threshold (H-Net) of a trained model and measure bpb (Figure 2).

4 Experimental setup

Data. FineWeb-Edu [12] `sample-10BT`, shards 0–1, as raw UTF-8 bytes (6.9 GB train; the last 1% of documents, 71 MB, for validation). Each run trains on 9,000 steps $\times$ 32 sequences $\times$ 2,048 bytes $\approx$ 590M bytes, and is evaluated on 100 batches of the validation set; the same seed implies the same data order in every arm.

Re-implementation ("ours"). A pure-PyTorch 1-stage hierarchy: byte embedding; encoder of 4 Transformer layers (d=512, 128-byte sliding-window attention); a chunker; main network of 12 Transformer layers (d=768, full causal attention); de-chunking; residual; decoder of 4 Transformer layers (d=512, 128-byte window); $\approx$150M parameters. AdamW, learning rate 10^{-3} with cosine decay and 1k warmup steps, main-network learning rate $\times 0.3$ (analogous to H-Net's per-stage multipliers), bf16, target compression N=4. All chunkers pass a causality test that perturbs future bytes and checks that earlier logits are unchanged.

Official code. The released `hnet` package, unmodified except for routing overrides for the "fixed", "random" and anticipatory variants: Mamba-2$\times$4 encoder and decoder (d=512), Transformer$\times$12 main network (d=768, FFN 2048), $\approx$100M parameters, the official load-balancing loss (α=0.03, N=4) and per-stage learning-rate multipliers $[2.0, 1.0]$ on a base rate of $5 \cdot 10^{-4}$. Identical data, steps and evaluation. "Fixed" forces a boundary every 4 bytes (the EMA then reduces to repetition and the straight-through factor to 1).

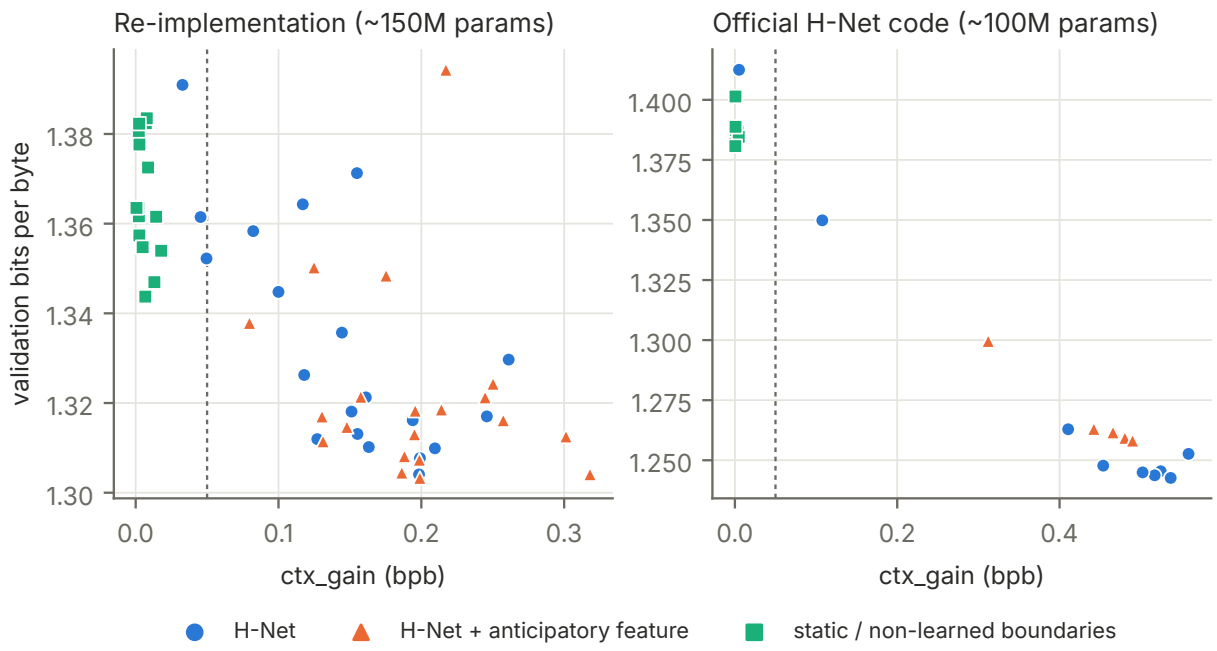


Figure 1: Final validation bpb against CTX_GAIN for every valid run (58 ours, 22 official). Static or non-learned boundaries (fixed stride, random boundaries, fixed stride with H-Net's EMA de-chunking, anticipatory boundaries without a learned router, and anti-collapse variants) all sit at CTX_GAIN ≈ 0. Dashed line: collapse threshold 0.05. Across all runs, $r(bpb, \text{CTX_GAIN}) = -0.91$.

Compute. H100 80GB PCIe (some official runs on A100), one or two runs per GPU, rented on demand; $\sim$1 h per run.

5 Results

5.1 Static chunking collapses; H-Net usually does not

Table 1 and Figure 1 summarise the main comparison. In our re-implementation, every run with static boundaries ended with CTX_GAIN ≤ 0.018 (17/17 runs across eight configurations), at 1.34–1.38 bpb. As these collapsed runs show, the 128-byte-window encoder and decoder alone form a byte LM of about this quality; the 12-layer main network adds nothing. Matching H-Net's realised compression with a 3.6-byte stride did not change this, nor did giving fixed boundaries H-Net's learned EMA de-chunking and straight-through gate ("fixed + H-Net de-chunking"), which shows that H-Net's de-chunker is not what keeps the main network alive.

The official implementation behaves the same way: official fixed-stride runs reach 1.38–1.40 bpb with CTX_GAIN $\leq$ 0.004, whereas the seven non-collapsed official H-Net seeds reach 1.243–1.263 bpb with CTX_GAIN of 0.41–0.56. When it works, the official model uses its main network far more than our re-implementation does (CTX_GAIN ≈ 0.5 vs. 0.1–0.3), and is correspondingly better (1.25 vs. $\approx$1.31 bpb).

H-Net's advantage tracks main-network use. Across all runs bpb and CTX_GAIN are strongly anti-correlated ($r = -0.91$; within the two H-Net arms of our re-implementation alone $r = -0.52$, where a few poor runs are not collapses). In this regime, the gap between H-Net and fixed-stride chunking (0.04 bpb ours, 0.11 bpb official) is therefore best described as the gap between a hierarchy that is used and one that is bypassed.

5.2 Collapse is stochastic, even for H-Net

With 20 seeds of our re-implementation, H-Net's final bpb ranges from 1.304 to 1.391 (Figure 3); 3/20 runs collapsed (CTX_GAIN ≤ 0.05) and 7/20 ended above 1.34 bpb. Official H-Net shows the same behaviour: of nine seeds, seven reached 1.243–1.263 bpb (CTX_GAIN ≥ 0.41), seed 2 reached only 1.350 (CTX_GAIN $= 0.11$), and seed 4 collapsed

Table 1: Main results (final validation bpb, 590M training bytes). "Collapsed" = runs with CTX_GAIN ≤ 0.05. Compression is bytes per chunk (target 4).

impl.	chunking	seeds	mean bpb	range	CTX_GAIN range	collapsed	compression
ours	fixed stride	2	1.369	1.357–1.381	0.002–0.003	2/2	4.00
ours	fixed stride, 3.6 bytes	2	1.366	1.355–1.378	0.003–0.004	2/2	3.60
ours	fixed + H-Net de-chunking	3	1.363	1.362–1.364	0.001–0.002	3/3	4.00
ours	H-Net	20	1.333	1.304–1.391	0.033–0.261	3/20	3.64
ours	H-Net + anticipatory	20	1.322	1.303–1.394	0.080–0.318	0/20	3.56
official	fixed stride	4	1.389	1.381–1.401	0.001–0.004	4/4	4.00
official	random boundaries	3	1.385	1.382–1.387	0.000–0.005	3/3	3.99
official	H-Net	9	1.278	1.243–1.413	0.005–0.559	1/9	3.46
official	H-Net + anticipatory	6	1.267	1.258–1.300	0.312–0.490	0/6	2.94

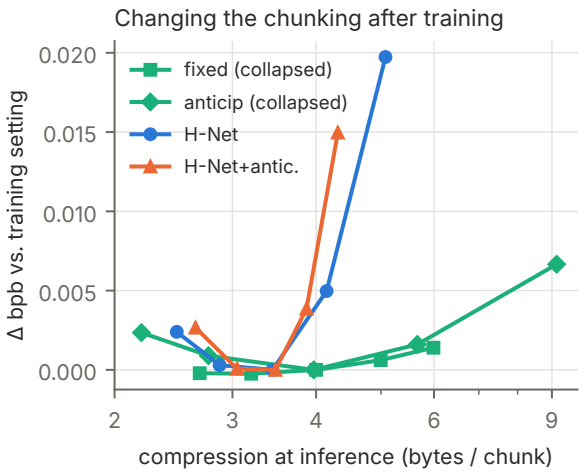


Figure 2: Changing the compression of a trained model at inference (ours). Collapsed models (fixed stride, anticipatory boundaries without a learned router) are almost insensitive to the chunking: fixed-stride bpb changes by at most 0.0016 between 2.7 and 6.0 bytes per chunk. Models whose main network is used (H-Net, H-Net + anticipatory) degrade quickly away from their training compression.

completely (1.413 bpb, CTX_GAIN $= 0.005$), ending *worse* than fixed-stride chunking. Single-seed comparisons between hierarchical byte LMs can therefore differ by more than the effects being compared.

5.3 Mechanism: content-dependent boundaries, decided early

Random boundaries collapse too. To separate "boundaries that vary" from "boundaries that depend on content", we trained the official architecture with boundaries drawn independently with probability $1/4$ at every byte (same compression as fixed stride, but irregular chunks). All three random-boundary runs collapsed (CTX_GAIN ≤ 0.005, 1.382–1.387 bpb), indistinguishable from fixed stride. Together with the re-implementation result that H-Net's de-chunking alone does not help static boundaries, this points at the *content-dependent, learned* placement of boundaries as what keeps the main network in use.

Collapse is decided early. Logging CTX_GAIN during training (Figure 4) shows two clean regimes. Runs whose main network is eventually used have CTX_GAIN $= 0.03$–0.13 already at step 1,000 (the end of learning-rate warm-up) and grow steadily; collapsed runs are at CTX_GAIN ≈ 0 by step 1,000 and stay there for the remaining 8,000 steps. Interventions against collapse therefore have to act at the very start of training. (Our residual warm-up, Section 5.4, acted over the first 3,000 steps and did not suffice for static boundaries.) A plausible account, which we have not tested directly, is a race at initialisation: the encoder–residual–decoder path learns local byte statistics quickly, after which the gradient reaching the main network through a decoder that already predicts well is too weak or too noisy for it to start carrying information, unless chunk boundaries that align with content make the main network's inputs informative early.

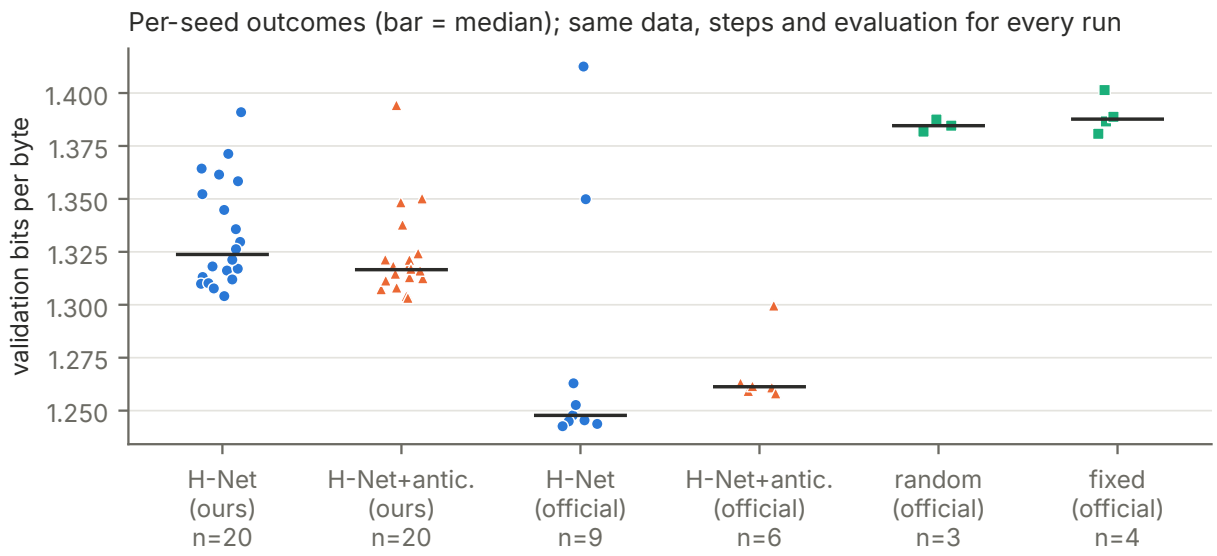


Figure 3: Per-seed final bpb. Ours: most H-Net seeds reach 1.30–1.33 bpb, but several land 0.04–0.06 bpb worse, mostly with a collapsed or partially collapsed main network. Official H-Net: seven seeds at 1.24–1.26 bpb, one partially collapsed (1.350, CTX_GAIN = 0.11), one fully collapsed (1.413, CTX_GAIN = 0.005). Random and fixed boundaries (official code) all collapse.

Table 2: Interventions and new chunkers (ours, final bpb). None revived the main network of a static-boundary model; none beat H-Net.

variant	seeds	mean bpb	CTX_GAIN
fixed stride (reference)	2	1.369	0.002–0.003
+ residual warm-up (bypass scaled $0 \to 1$ over 3k steps)	2	1.350	0.013–0.018
+ LayerNorm on main-network input	2	1.349	0.005–0.007
+ main-network lr $\times 0.1$ (instead of $\times 0.3$)	2	1.370	0.002–0.003
anticipatory boundaries, no learned router	3	1.376	0.007–0.014
+ residual warm-up	1	1.373	0.009
H-Net + residual warm-up (a collapsing seed)	1	1.405	0.083

5.4 What did not fix collapse

Anti-collapse interventions. Scaling the residual bypass from 0 to 1 over the first 3k steps, normalising the main network's input, or lowering the main network's learning rate further each left fixed-stride models collapsed (Table 2); the first two improved bpb by $\approx$0.02 without increasing CTX_GAIN meaningfully, i.e. they helped the outer networks, not the hierarchy. Residual warm-up also did not rescue an H-Net seed that collapses without it.

New chunking mechanisms. Before identifying collapse we designed and evaluated several mechanisms (definitions in Appendix A): continuous integrate-and-fire chunking [6] with exact compression control and three variants; equal-information chunking (fire when accumulated next-byte surprisal of a detached probe reaches a running budget); anticipatory chunking (end a chunk where the predictive entropy of the next byte is high; 66% of its boundaries fall on spaces or punctuation); H-Net with a realised-surprisal feature; and fixed boundaries with H-Net's learned de-chunking. Integrate-and-fire, equal-information and realised-surprisal routing were only evaluated in the CPU study and in GPU runs with the bypassed configuration of Appendix B, so we do not draw conclusions about them beyond "no evidence of an improvement". In the valid GPU setting none of the mechanisms outperformed H-Net, and every mechanism without a learned, content-dependent router collapsed.

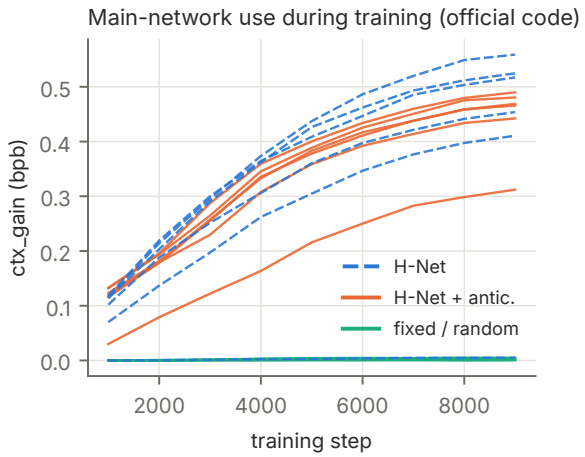


Figure 4: CTX_GAIN measured every 1,000 steps (official code; 16 runs). Every run that ends with a used main network already has CTX_GAIN ≥ 0.03 at step 1,000 and increases monotonically. Every fixed-stride and random-boundary run, and the collapsing H-Net seed (dashed line at 0), is at CTX_GAIN ≈ 0 from step 1,000 onwards and never recovers.

5.5 Anticipatory routing: no collapses observed, no significant gain

Our most promising variant keeps H-Net's router and adds one feature: the standardised predictive entropy $\hat{H}_t$ of the next byte, computed by a small next-byte probe on detached encoder states (trained by its own cross-entropy, with running normalisation statistics so that it is causal),

$$p_t = \text{clip}\Big(\tfrac{1}{2}\big(1-\cos(W_q h_{t-1}, W_k h_t)\big) + \gamma \hat{H}_t,\ 0, 1\Big),$$

with learned γ (initialised to 0.1). The intent is to call the main network just before bytes that the encoder finds hard to predict. Over 20 seeds it never collapsed (CTX_GAIN ≥ 0.08), ended above 1.34 bpb in 3/20 seeds (H-Net: 7/20), and averaged 1.322 vs. 1.333 bpb for H-Net (Table 1). None of these differences is significant: Welch t-test $p = 0.15$, Mann–Whitney $p = 0.15$, seed-paired t-test $p = 0.20$, Levene $p = 0.19$, Fisher's exact test on collapse counts $p = 0.23$. An interim analysis with 14 vs. 16 seeds showed a significant variance reduction (Levene $p = 0.046$) that did not survive the remaining seeds; one anticipatory seed ended at 1.394 bpb *without* collapsing (CTX_GAIN $= 0.22$).

In the official code (6 seeds), the anticipatory feature again never collapsed (CTX_GAIN ≥ 0.31; mean 1.267 vs. 1.278 for nine H-Net seeds, Welch $p = 0.62$). However, its non-collapsed runs (1.258–1.300 bpb) are slightly *worse* than H-Net's seven non-collapsed runs (1.243–1.263), and it settles at a lower compression (2.94 vs. 3.46 bytes per chunk), i.e. it spends roughly 18% more main-network compute. Pooling both implementations, collapse (CTX_GAIN ≤ 0.05) occurred in 0/26 anticipatory runs vs. 4/29 H-Net runs (Fisher $p = 0.11$). We therefore report a null result: a possible reduction in collapse risk that our sample cannot establish, and no improvement when H-Net trains normally.

6 Related work

Hierarchical and pooled Transformers shorten sequences inside the network [3, 9]; MEGABYTE [14] and SpaceByte [13] apply fixed or rule-based patching to bytes, BLT [11] uses entropy-based patches from a separately trained byte LM, and dynamic token pooling [10], MrT5 [8], H-Net [7], H-Net++ [15] and ByteFlow [5] learn where to segment or merge. Continuous integrate-and-fire [6] comes from speech recognition. The failure mode we describe resembles posterior collapse in VAEs [2], where a powerful decoder ignores a latent pathway that the objective does not force it to use; here the ignored pathway is the chunk-level main network, and the shortcut is the encoder-to-decoder residual.

7 Limitations

Our models are small (100–150M parameters) and trained on 590M bytes, far below the scale of the H-Net paper; the residual shortcut may matter less when encoder and decoder are relatively weaker or training is longer. We use one dataset, one-stage hierarchies, and target compression 4, and a short training budget; collapse might be escaped later in much longer runs, although we saw no recovery in 8,000 steps after it occurred. Official-code results use 3–9 seeds per

arm. Our re-implementation differs from the official code (Transformer instead of Mamba-2 encoder/decoder) and is weaker in absolute terms. We did not tune the load-balancing weight or learning rates of the official code for this smaller model. CTX_GAIN measures dependence on the main network's chunk-specific content under mean replacement; it is a diagnostic, not a complete account of what the main network computes.

8 Conclusion

In hierarchical byte-level LMs the main network can be bypassed without any visible failure in the loss. At our scale this happened in every model with static chunk boundaries, in both a re-implementation and the official H-Net code, and occasionally to H-Net itself; H-Net's advantage over fixed chunking was largely the difference between a used and an unused hierarchy. We recommend that work on hierarchical byte-level models report CTX_GAIN or a similar bypass check, test sensitivity to the chunking at inference, and report multiple seeds.

Reproducibility. Code, run configurations, all per-run results and the scripts that produce every table and figure are in the accompanying repository (`research/dcbench`).

References

[1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[2] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. *arXiv preprint arXiv:1511.06349*, 2015.

[3] Zihang Dai, Guokun Lai, Yiming Yang, and Quoc V. Le. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. *arXiv preprint arXiv:2006.03236*, 2020.

[4] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[5] Chunyuan Deng, Sanket Lokegaonkar, Colin Lockard, Besnik Fetahu, Nasser Zalmout, and Xian Li. ByteFlow: Language modeling through adaptive byte compression without a tokenizer. *arXiv preprint arXiv:2603.03583*, 2026. ICLR 2026.

[6] Linhao Dong and Bo Xu. CIF: Continuous integrate-and-fire for end-to-end speech recognition. *arXiv preprint arXiv:1905.11235*, 2019.

[7] Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*, 2025.

[8] Julie Kallini, Shikhar Murty, Christopher D. Manning, Christopher Potts, and Róbert Csordás. MrT5: Dynamic token merging for efficient byte-level language models. *arXiv preprint arXiv:2410.20771*, 2024.

[9] Piotr Nawrot, Szymon Tworkowski, Michał Tyrolski, Łukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language models. *arXiv preprint arXiv:2110.13711*, 2021.

[10] Piotr Nawrot, Jan Chorowski, Adrian Łańcucki, and Edoardo M. Ponti. Efficient transformers with dynamic token pooling. *arXiv preprint arXiv:2211.09761*, 2022.

[11] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[12] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[13] Kevin Slagle. SpaceByte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

[14] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. *arXiv preprint arXiv:2305.07185*, 2023.

[15] Mehrdad Zakershahrak and Samira Ghodratnama. H-Net++: Hierarchical dynamic chunking for tokenizer-free language modelling in morphologically-rich languages. *arXiv preprint arXiv:2508.05628*, 2025.

A Chunking mechanisms

All mechanisms share the encoder, main network, decoder and residual path; only down-sampling and up-sampling differ. Byte t may only read chunks completed at or before t.

- **Fixed**: a boundary every N bytes (fractional N allowed via $\lfloor t/N \rfloor$).
- **H-Net** [7]: cosine router, EMA de-chunking, straight-through confidence, load-balancing loss (Section 2).
- **Integrate-and-fire (IF)**: each byte emits $a_t \in (0,1)$, $C_t = \sum_{s \le t} a_s$; a chunk fires when C_t crosses an integer, so the number of chunks is exactly $\lfloor \sum a_t \rfloor$. The chunk vector is the a-weighted average of its bytes (`if`) or, in the forward pass, the state of the firing byte with gradients through the pooling weights (`if_last`).
- **Equal-information**: $a_t = \min(1, s_t/B)$ with s_t the surprisal of byte t under a detached next-byte probe and B a running budget $\approx N \cdot \mathbb{E}[s]$.
- **Anticipatory boundaries**: boundary at t when the probe's predictive entropy of byte $t{+}1$ exceeds a threshold adjusted online to keep the boundary rate at $1/N$.
- **H-Net + realised surprisal / + anticipatory**: H-Net's boundary probability plus γ times a standardised surprisal (of x_t) or predictive-entropy (of x_{t+1}) feature.
- **Fixed + H-Net de-chunking**: fixed boundaries, but H-Net's learned p drives the EMA de-chunking and the straight-through gate.
- **Random** (official code): boundaries drawn independently with probability $1/N$.

B Earlier experiments

A CPU study (12.5M parameters, 6.5 MB of English text, 1,500 steps) and three initial GPU waves preceded the results above. In the CPU study no mechanism separated from fixed-stride chunking beyond seed noise ($\approx$0.01 bpb), and an early single-seed "win" of integrate-and-fire chunking (0.09 bpb) on a distribution-shifted validation split did not replicate on a clean split. The first three GPU waves used full causal attention in the encoder and decoder and a main-network learning rate of 10^{-3}; a compression sweep showed bpb unchanged to four decimals between 2.7 and 6 bytes per chunk, which led us to the CTX_GAIN diagnostic. We exclude those waves from all conclusions.

C Compute

All runs used rented spot GPUs (NVIDIA H100 80GB PCIe; three official runs on A100 80GB), one or two runs per GPU, about one hour per run. Including the CPU study, the excluded early GPU waves and failed installation attempts, the project used roughly 230 USD of GPU time.

D Use of AI assistance

The experiments, analysis code and a first draft of this paper were produced with the help of an AI coding agent operating under the authors' direction; all numbers are generated by the scripts in the repository from the logged runs.